React + Node.js Integration Guide

Building full-stack applications with a React frontend and a Node.js/Express backend

Table of contents

1. Introduction and project setup

2. Setting up the Node.js and Express backend

3. Setting up the React frontend

4. Connecting React to an Express API

5. Handling forms and user input

6. Authentication with JWT

7. Protecting routes on frontend and backend

8. State management: Context API and Redux

9. Working with MongoDB and Mongoose

10. File uploads between React and Node.js

11. Real-time features with WebSockets

12. Error handling and validation

13. Testing the integration

14. Environment configuration and CORS

15. Deployment strategies

16. Performance and production checklist

17. Caching with Redis

18. Background jobs and queues

19. Documenting the API

20. Logging and monitoring

21. Worked example: adding a comments feature end to end

1. Introduction and project setup

This guide walks through building a full-stack application where a React single-page application communicates with a Node.js and Express backend over a REST API, with a MongoDB database for persistence. The goal is to cover the integration points that trip people up in practice: how the two projects are structured, how requests flow between them, how authentication is shared across the boundary, and how the whole thing is deployed.

We assume Node.js 18 or later, npm or yarn, and a MongoDB instance (local or Atlas). The project is organized as two separate applications living in one repository: a client folder for the React app (created with Vite) and a server folder for the Express API. Keeping them separate, rather than one monolithic app, makes the deployment story simpler and mirrors how most production teams structure things.

```
project-root/
  client/           # React app (Vite)
    src/
    package.json
  server/           # Express API
    src/
    package.json
  package.json       # optional root scripts (concurrently, etc.)
```

A root-level package.json with the concurrently package lets you run both dev servers with a single command during local development, which we set up at the end of this chapter.

Why separate frontend and backend?

Separating the client and server gives you independent deploy pipelines, independent scaling, and the option to swap either side later without touching the other. The backend exposes a versioned REST API; the frontend is just one consumer of that API and could, in principle, be replaced by a mobile app without any backend changes.

Initializing the backend

```
mkdir server && cd server
npm init -y
npm install express cors dotenv mongoose
npm install -D nodemon
```

Initializing the frontend

```
npm create vite@latest client -- --template react
cd client
npm install
npm install axios
```

With both projects scaffolded, the next two chapters build out the backend API and the frontend shell before we connect them.

2. Setting up the Node.js and Express backend

The backend is a small Express application exposing a REST API under /api. We start with a minimal server, then add routing, middleware, and a database connection.

Minimal server

```
// server/src/index.js
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok' });
});

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log(`API running on port ${PORT}`));
```

The cors middleware is essential here: without it, a React app served from a different origin (a different port during development, a different domain in production) will have its requests blocked by the browser's same-origin policy. We cover CORS configuration in more depth in chapter 14.

Organizing routes

As the API grows, routes should be split into their own files rather than living in index.js. A common pattern is one router per resource.

```
// server/src/routes/tasks.js
const express = require('express');
const router = express.Router();
const Task = require('../models/Task');

router.get('/', async (req, res) => {
  const tasks = await Task.find();
  res.json(tasks);
});

router.post('/', async (req, res) => {
  const task = await Task.create(req.body);
  res.status(201).json(task);
});

module.exports = router;

// server/src/index.js (excerpt)
const taskRoutes = require('./routes/tasks');
app.use('/api/tasks', taskRoutes);
```

Connecting to MongoDB

```
// server/src/db.js
const mongoose = require('mongoose');

async function connectDB() {
  await mongoose.connect(process.env.MONGO_URI);
  console.log('MongoDB connected');
}

module.exports = connectDB;
```

Call connectDB() once at startup, before app.listen. Mongoose queues operations until the connection is established, so in practice requests that arrive during startup will simply wait rather than fail, but it is still good hygiene to connect before accepting traffic in production.

Middleware order matters

Express middleware runs in the order it is registered. Body-parsing middleware (express.json()) must be registered before any route that reads req.body. Authentication middleware must run before the route handlers it protects. A common bug is registering error-handling middleware too early, which causes it to be skipped entirely — error handlers must be the last app.use() call, after all routes.

3. Setting up the React frontend

With Vite, the React app starts as a single App component rendering into a div in index.html. For a real application you will want a router, a place to keep API call logic, and a consistent way to handle loading and error states.

Project structure

```
client/src/
  api/            # axios instance + endpoint functions
  components/
  pages/
  hooks/
  App.jsx
  main.jsx
```

A shared axios instance

Rather than calling axios.get(...) with a full URL in every component, create a single configured instance so the base URL and headers live in one place.

```
// client/src/api/client.js
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:4000/api',
});

export default api;
```

import.meta.env.VITE_API_URL reads a Vite environment variable, which lets the same build point at different backends in development, staging, and production without changing code. Vite only exposes variables prefixed with VITE_ to client code, which is a deliberate safety boundary — anything without that prefix stays server-side only, relevant if you ever run Vite's SSR mode.

A first API call

```
// client/src/pages/TaskList.jsx
import { useEffect, useState } from 'react';
import api from '../api/client';

export default function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get('/tasks')
      .then(res => setTasks(res.data))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading...</p>;
```

```
  return (
    <ul>
      {tasks.map(t => <li key={t._id}>{t.title}</li>)}
    </ul>
  );
}
```

This pattern — a loading flag, a data state, and an effect that fires once on mount — is the backbone of nearly every React-to-API integration. The next chapter generalizes it into a reusable hook and covers the failure cases this example glosses over.

4. Connecting React to an Express API

The TaskList example in the previous chapter works but ignores errors and re-fetches. A more robust pattern wraps the request in a custom hook that tracks loading, data, and error state together.

```
// client/src/hooks/useFetch.js
import { useEffect, useState, useCallback } from 'react';
import api from '../api/client';

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  const refetch = useCallback(() => {
    setLoading(true);
    api.get(url)
      .then(res => setData(res.data))
      .catch(err => setError(err))
      .finally(() => setLoading(false));
  }, [url]);

  useEffect(() => { refetch(); }, [refetch]);

  return { data, error, loading, refetch };
}
```

Every component that needs server data can now call useFetch('/tasks') and get consistent loading and error handling for free. The refetch function is exposed so components can manually trigger a re-fetch after a mutation, such as creating a new task.

Sending data: POST requests

```
async function createTask(title) {
  const res = await api.post('/tasks', { title });
  return res.data;
}
```

Handling network errors gracefully

A request can fail for several distinct reasons, and the UI should distinguish them: the server is unreachable (network error, err.request is set but err.response is not), the server responded with a 4xx (client error, likely a validation problem, shown to the user), or the server responded with a 5xx (server error, generally not the user's fault and worth a generic retry message). Axios makes this distinction available on the caught error object, and it is worth writing a small helper to classify errors consistently across the app rather than repeating the same if/else in every catch block.

```
function classifyError(err) {
  if (!err.response) return 'network';
  if (err.response.status >= 500) return 'server';
  return 'client';
```

```
}
```

5. Handling forms and user input

Forms are where the frontend and backend contracts matter most, because validation logically needs to exist on both sides: client-side for immediate feedback, server-side because the client can never be trusted to be the only gatekeeper.

Controlled form components

```
function NewTaskForm({ onCreated }) {
  const [title, setTitle] = useState('');
  const [submitting, setSubmitting] = useState(false);

  async function handleSubmit(e) {
    e.preventDefault();
    setSubmitting(true);
    try {
      const task = await createTask(title);
      onCreated(task);
      setTitle('');
    } catch (err) {
      alert('Could not create task');
    } finally {
      setSubmitting(false);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={title} onChange={e => setTitle(e.target.value)} />
      <button disabled={submitting}>Add</button>
    </form>
  );
}
```

Disabling the submit button while submitting is a small detail that prevents a common bug: users double-clicking submit on a slow connection and creating duplicate records. The backend should also guard against this with idempotency keys or unique constraints where it matters, since the frontend guard can always be bypassed.

Server-side validation

A library like Joi, zod, or express-validator lets the backend define a schema once and validate incoming bodies before they reach business logic, returning a structured 422 response the frontend can map back onto specific form fields.

```
const { z } = require('zod');

const taskSchema = z.object({
  title: z.string().min(1).max(200),
});

router.post('/', (req, res, next) => {
```

```
  const parsed = taskSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(422).json({ errors: parsed.error.flatten() });
  }
  next();
}, async (req, res) => {
  const task = await Task.create(parsed.data);
  res.status(201).json(task);
});
```

6. Authentication with JWT

JSON Web Tokens are the most common way to authenticate a React SPA against a Node.js API. The flow: the user logs in with credentials, the server verifies them and issues a signed token, the client stores the token and attaches it to subsequent requests, and the server verifies the token's signature on each protected request.

Issuing a token

```
// server/src/routes/auth.js
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
  const token = jwt.sign({ sub: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
  res.json({ token });
});
```

Storing the token on the client

The two common options are localStorage and an httpOnly cookie. localStorage is simple and works well with a pure REST API, but is readable by any JavaScript running on the page, which makes it vulnerable to XSS. An httpOnly cookie set by the server is not readable by JavaScript at all, which closes that particular hole, but requires the server to set SameSite and CORS credentials correctly, and requires the frontend to send withCredentials: true on requests. For most small-to-medium apps, localStorage with careful XSS hygiene (escaping all user-generated content, a strict Content-Security-Policy) is an acceptable and much simpler starting point; move to httpOnly cookies once the app handles sensitive data at scale.

```
// client/src/api/client.js (updated)
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Verifying the token on the backend

```
// server/src/middleware/auth.js
const jwt = require('jsonwebtoken');

function requireAuth(req, res, next) {
  const header = req.headers.authorization;
  if (!header) return res.status(401).json({ error: 'No token' });
  try {
    const payload = jwt.verify(header.split(' ')[1], process.env.JWT_SECRET);
```

```
    req.userId = payload.sub;
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}

module.exports = requireAuth;
```

7. Protecting routes on frontend and backend

Authentication has to be enforced on both sides. The backend middleware from the previous chapter is the real security boundary — it is the only thing an attacker cannot bypass. Frontend route guards exist purely for user experience: redirecting an unauthenticated user to the login page before they see a flash of protected content, or before they waste a request that will just come back 401.

Applying backend middleware selectively

```
router.get('/tasks', requireAuth, async (req, res) => {
  const tasks = await Task.find({ owner: req.userId });
  res.json(tasks);
});
```

Scoping the query to req.userId, set by the auth middleware from the verified token, is what actually enforces that users can only see their own data. Trusting a userId sent in the request body instead of the token would let any client claim to be any user.

A frontend route guard

```
function ProtectedRoute({ children }) {
  const token = localStorage.getItem('token');
  if (!token) return <Navigate to="/login" replace />;
  return children;
}

// usage in the router
<Route path="/tasks" element={<ProtectedRoute><TaskList /></ProtectedRoute>} />
```

Handling token expiry

When a request comes back 401 because the token expired, the cleanest pattern is a response interceptor that clears the stored token and redirects to login in one place, rather than handling it in every component that makes a request.

```
api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);
```

8. State management: Context API and Redux

Once authentication state, the current user, and cached server data all need to be shared across many components, prop drilling stops scaling. React's built-in Context API is enough for a lot of applications; Redux (or Redux Toolkit) earns its complexity when you have many independent pieces of shared state updated from many places, or when you want time-travel debugging and a strict, testable reducer pattern.

An auth context

```
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  async function login(email, password) {
    const { data } = await api.post('/auth/login', { email, password });
    localStorage.setItem('token', data.token);
    setUser(data.user);
  }

  function logout() {
    localStorage.removeItem('token');
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

Redux Toolkit for server data

Redux Toolkit's createAsyncThunk pairs naturally with an Express API: a thunk dispatches pending, fulfilled, or rejected actions around an axios call, and a slice's extraReducers updates state accordingly. This is most valuable when the same server data (a list of tasks, say) is read and mutated from many unrelated parts of the UI, and you want a single source of truth rather than each component re-fetching independently.

```
export const fetchTasks = createAsyncThunk('tasks/fetch', async () => {
  const res = await api.get('/tasks');
  return res.data;
});

const tasksSlice = createSlice({
  name: 'tasks',
  initialState: { items: [], status: 'idle' },
  reducers: {},
```

```
  extraReducers: builder => {
    builder
      .addCase(fetchTasks.pending, state => { state.status = 'loading'; })
      .addCase(fetchTasks.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.items = action.payload;
      });
  },
});
```

A rule of thumb: reach for Context for low-frequency, app-wide values like the current user or theme. Reach for Redux Toolkit (or a library like React Query/TanStack Query) once you are managing server-cache concerns — refetching, invalidation, optimistic updates — since those tools were purpose-built for that problem and Context was not.

9. Working with MongoDB and Mongoose

Mongoose sits between Express route handlers and MongoDB, giving you schemas, validation, and a query API on top of the raw driver. Defining schemas explicitly, even though MongoDB itself is schemaless, catches a large class of bugs before they reach the database.

```
// server/src/models/Task.js
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true, maxlength: 200 },
  completed: { type: Boolean, default: false },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
}, { timestamps: true });

module.exports = mongoose.model('Task', taskSchema);
```

Indexes matter for API response time

Every query the API runs against a user-scoped field, like owner in the schema above, should have an index on that field, or the query degrates to a full collection scan as data grows. This is invisible in development with a handful of documents and becomes a real production incident once a collection reaches hundreds of thousands of rows.

```
taskSchema.index({ owner: 1, createdAt: -1 });
```

Population and the N+1 problem

Mongoose's populate() resolves referenced documents (like owner) in a single additional query rather than one query per document, which avoids the classic N+1 problem you'd get by naively looping and calling findById inside a map.

```
const tasks = await Task.find({ owner: req.userId }).populate('owner', 'name email');
```

Transactions for multi-document writes

When a single logical operation needs to update more than one collection consistently, such as decrementing an inventory count while creating an order, Mongoose exposes MongoDB's session-based transactions. These require a replica set (Atlas provides this by default); a standalone local MongoDB instance needs to be configured as a single-node replica set for transactions to work during local development.

10. File uploads between React and Node.js

File uploads are one of the few integration points where you cannot simply send JSON — the request needs to be multipart/form-data, and both sides need to agree on that.

Frontend: building a FormData request

```
async function uploadAvatar(file) {
  const formData = new FormData();
  formData.append('avatar', file);
  const res = await api.post('/users/avatar', formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  });
  return res.data;
}

// triggered from an <input type="file" onChange={e => uploadAvatar(e.target.files[0])} />
```

Backend: parsing multipart data with multer

```
const multer = require('multer');
const upload = multer({ dest: 'uploads/', limits: { fileSize: 5 * 1024 * 1024 } });

router.post('/users/avatar', requireAuth, upload.single('avatar'), async (req, res) => {
  await User.findByIdAndUpdate(req.userId, { avatarPath: req.file.path });
  res.json({ path: req.file.path });
});
```

The limits option is worth setting explicitly — an unbounded upload endpoint is a straightforward denial-of-service vector. In production, uploaded files are typically streamed to object storage (S3 or equivalent) rather than kept on the API server's local disk, since local disk does not survive a redeploy and does not scale across multiple API instances.

11. Real-time features with WebSockets

REST works well for request/response interactions but not for the server pushing updates to the client unprompted — a chat message arriving, a task being updated by another user. Socket.IO is the most common choice for layering this onto an existing Express app.

```
// server/src/index.js (excerpt)
const http = require('http');
const { Server } = require('socket.io');

const server = http.createServer(app);
const io = new Server(server, { cors: { origin: process.env.CLIENT_URL } });

io.on('connection', socket => {
  socket.on('join-room', roomId => socket.join(roomId));
  socket.on('task-updated', (roomId, task) => {
    socket.to(roomId).emit('task-updated', task);
  });
});

server.listen(PORT);

// client/src/hooks/useSocket.js
import { io } from 'socket.io-client';
import { useEffect, useRef } from 'react';

export function useSocket(roomId, onTaskUpdated) {
  const socketRef = useRef(null);

  useEffect(() => {
    const socket = io(import.meta.env.VITE_API_URL);
    socketRef.current = socket;
    socket.emit('join-room', roomId);
    socket.on('task-updated', onTaskUpdated);
    return () => socket.disconnect();
  }, [roomId]);
}
```

Note that server.listen replaces app.listen once Socket.IO is attached — the HTTP server instance is shared between Express's request handling and the WebSocket upgrade handshake. A common mistake is leaving app.listen in place alongside a separate server.listen, which silently starts two servers on different ports and leaves Socket.IO unreachable from the client's configured URL.

12. Error handling and validation

A consistent error shape returned by the API makes frontend error handling dramatically simpler. Rather than each endpoint inventing its own error format, define one shape and use it everywhere.

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Title is required", "fields": { "title": "req
```

A centralized Express error handler

```
// server/src/middleware/errorHandler.js
function errorHandler(err, req, res, next) {
  const status = err.status || 500;
  res.status(status).json({
    error: {
      code: err.code || 'INTERNAL_ERROR',
      message: status === 500 ? 'Something went wrong' : err.message,
    },
  });
}

module.exports = errorHandler;

// server/src/index.js — registered LAST, after all routes
app.use(errorHandler);
```

Route handlers can then simply throw an error with a status and code, and rely on this single handler rather than repeating try/catch and response-shaping in every route. With async route handlers, remember that a thrown error inside an async function does not automatically reach Express's error handler unless it's caught and passed to next(err) — either wrap handlers in a small async-catching utility, or use Express 5, where this is handled automatically.

```
function asyncHandler(fn) {
  return (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
}

router.get('/tasks/:id', asyncHandler(async (req, res) => {
  const task = await Task.findById(req.params.id);
  if (!task) { const e = new Error('Not found'); e.status = 404; throw e; }
  res.json(task);
}));
```

Mapping errors on the frontend

With a consistent error shape, a single response interceptor or a shared catch helper can extract error.message and error.fields, show a toast for general errors, and attach field-level errors directly to form inputs.

13. Testing the integration

Three layers of tests give confidence at different costs. Unit tests for backend logic (Jest, mocking Mongoose models) are cheap and fast. Integration tests that spin up the real Express app against an in-memory MongoDB (mongodb-memory-server) verify routes end to end without needing a live database. End-to-end tests (Playwright or Cypress) drive the real React app against a real running backend and are the slowest but catch integration bugs the other two layers cannot, such as a frontend field name that no longer matches the backend's expected body shape.

Backend integration test example

```
const request = require('supertest');
const app = require('../src/app');

describe('POST /api/tasks', () => {
  it('creates a task', async () => {
    const res = await request(app)
      .post('/api/tasks')
      .set('Authorization', `Bearer ${testToken}`)
      .send({ title: 'Write docs' });
    expect(res.status).toBe(201);
    expect(res.body.title).toBe('Write docs');
  });
});
```

Frontend component test with mocked API

```
import { render, screen, fireEvent } from '@testing-library/react';
import { vi } from 'vitest';
import api from '../api/client';
import NewTaskForm from './NewTaskForm';

vi.mock('../api/client');

test('submits a new task', async () => {
  api.post.mockResolvedValue({ data: { _id: '1', title: 'Test' } });
  render(<NewTaskForm onCreated={vi.fn()} />);
  fireEvent.change(screen.getByRole('textbox'), { target: { value: 'Test' } });
  fireEvent.click(screen.getByText('Add'));
  expect(api.post).toHaveBeenCalledWith('/tasks', { title: 'Test' });
});
```

Mocking the shared axios client (rather than mocking fetch or a specific endpoint globally) keeps frontend tests decoupled from the backend actually running, while the shape of what's mocked stays close to what the real API returns — a contract worth keeping in sync manually, or generating from an OpenAPI spec if the project has one.

14. Environment configuration and CORS

Development and production need different API URLs, different CORS origins, and different secrets. Both the client and server read these from environment variables, but the mechanics differ.

Backend environment variables

```
# server/.env
PORT=4000
MONGO_URI=mongodb://localhost:27017/myapp
JWT_SECRET=change-me
CLIENT_URL=http://localhost:5173
```

Frontend environment variables

```
# client/.env
VITE_API_URL=http://localhost:4000/api
```

Vite bakes VITE_-prefixed variables into the built JavaScript bundle at build time, not at runtime — there is no server reading a .env file when a static React build is served. This means a production build needs its own .env.production (or CI-provided build-time variables) and cannot have its API URL changed after the fact without rebuilding, unless you introduce a small runtime-config.json fetched on load, a common workaround for apps deployed as static files behind a CDN.

Configuring CORS precisely

```
app.use(cors({
  origin: process.env.CLIENT_URL,
  credentials: true,
}));
```

origin: '*' is fine for a fully public, unauthenticated API, but must not be combined with credentials: true — browsers reject that combination outright, and for good reason: it would let any website read authenticated responses on a user's behalf. Set origin to the exact client URL(s) once cookies or authorization headers are involved.

15. Deployment strategies

Because the client and server are separate applications, they can be deployed to different platforms optimized for each. Two common patterns cover most cases.

Pattern A: static frontend + separate API host

The React build output (a folder of static HTML/JS/CSS) is deployed to a static host or CDN (Vercel, Netlify, S3+CloudFront). The Express API is deployed separately to a Node-friendly host (Render, Railway, Fly.io, an EC2 instance, or a container platform). This is the most common pattern and the one this guide has assumed throughout: two independent deployments, connected only by VITE_API_URL pointing at the API's public URL and CORS configured to allow the frontend's domain.

Pattern B: Express serving the React build

Alternatively, the Express server can serve the built React app as static files and let React Router handle client-side routing, collapsing everything into one deployable unit — simpler operationally, at the cost of coupling frontend and backend release cycles.

```
app.use(express.static(path.join(__dirname, '../../client/dist')));

app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, '../../client/dist/index.html'));
});
```

This catch-all route must be registered after all /api routes, or it will swallow API requests and return index.html instead of JSON, a very common and confusing bug when this pattern is set up in the wrong order.

A minimal CI pipeline

A typical GitHub Actions workflow runs backend tests, runs frontend tests and a production build, and only deploys if both succeed — catching integration breakage (a renamed field, a changed status code) before it reaches production rather than after.

16. Performance and production checklist

A final checklist of details that matter once the integration moves from a working demo to something serving real traffic.

- Enable gzip/br compression on the Express server (compression middleware) for JSON and static assets.
- Add rate limiting (express-rate-limit) on authentication endpoints specifically, not just globally.
- Set security headers with helmet() rather than configuring each header manually.
- Paginate any list endpoint that can grow unbounded — never return an entire collection by default.
- Add database indexes for every field used in a query filter, not just the ones used today.
- Log requests with correlation IDs so a frontend error report can be traced to the exact backend request.
- Set sensible axios timeouts on the frontend so a hung backend doesn't hang the UI indefinitely.
- Version the API (/api/v1/...) before the first breaking change, not after.
- Validate environment variables at startup and fail fast with a clear message if one is missing.
- Keep the JWT expiry short and pair it with a refresh token flow for anything beyond a small internal tool.

The remaining chapters cover topics that come up once an application grows past its first version: caching, background jobs, API documentation, monitoring, and a worked example that ties several pieces together.

17. Caching with Redis

Once an endpoint is expensive — an aggregation query, an external API call — caching the response keeps latency predictable under load. Redis is the standard choice because it is fast, supports expiry natively, and is easy to run alongside Node.js.

```
const redis = require('redis');
const client = redis.createClient({ url: process.env.REDIS_URL });
await client.connect();

async function getCachedTasks(userId) {
  const key = `tasks:${userId}`;
  const cached = await client.get(key);
  if (cached) return JSON.parse(cached);

  const tasks = await Task.find({ owner: userId });
  await client.set(key, JSON.stringify(tasks), { EX: 60 });
  return tasks;
}
```

The cache key must be invalidated (or simply left to expire) whenever the underlying data changes — creating, updating, or deleting a task for that user. A short TTL, as used above, is often a pragmatic middle ground: it bounds staleness without requiring precise invalidation logic on every write path.

Cache-aside vs write-through

The pattern above is cache-aside: the application checks the cache, and on a miss, reads from the database and populates the cache. Write-through caching, where every write updates the cache immediately, keeps the cache always fresh at the cost of more complex write paths. For most REST APIs backing a React frontend, cache-aside with a short TTL is simpler to reason about and sufficient.

18. Background jobs and queues

Some operations — sending a welcome email, resizing an uploaded image, generating a report — should not block the HTTP response the React app is waiting on. Offloading them to a queue keeps API response times fast and predictable.

```
// producer (inside a route handler)
const { Queue } = require('bullmq');
const emailQueue = new Queue('emails', { connection: redisConnection });

router.post('/users', async (req, res) => {
  const user = await User.create(req.body);
  await emailQueue.add('welcome', { userId: user._id });
  res.status(201).json(user);
});

// worker (a separate Node.js process)
const { Worker } = require('bullmq');

new Worker('emails', async job => {
  if (job.name === 'welcome') {
    await sendWelcomeEmail(job.data.userId);
  }
}, { connection: redisConnection });
```

The React app never talks to the queue directly — from its point of view, POST /users simply returns quickly. If the frontend needs to reflect the job's eventual outcome (the email was sent, the report is ready), that's typically surfaced later via a WebSocket event, as covered in chapter 11, or by the client polling a status endpoint.

19. Documenting the API

Once more than one frontend developer is consuming the API, or once the API is meant to outlive tribal knowledge, a machine-readable spec pays for itself. OpenAPI (formerly Swagger) is the standard format, and it can be generated from JSDoc-style comments directly above route handlers rather than maintained as a separate hand-written file.

```
/**
 * @openapi
 * /tasks:
 *   get:
 *     summary: List tasks for the authenticated user
 *     responses:
 *       200:
 *         description: A list of tasks
 */
router.get('/', requireAuth, asyncHandler(async (req, res) => {
  res.json(await Task.find({ owner: req.userId }));
}));
```

swagger-jsdoc collects these comments into a spec object, and swagger-ui-express serves an interactive documentation page from that spec at a route like /api/docs. Beyond human-readable docs, the same OpenAPI spec can generate a typed API client for the React app (via openapi-typescript-codegen or similar), which keeps request and response types in sync with the backend automatically rather than by convention.

20. Logging and monitoring

In production, console.log is not a logging strategy. A structured logger (pino or winston) that emits JSON lines makes logs searchable and lets you attach consistent metadata — a request ID, a user ID — to every line related to one request.

```
const pino = require('pino');
const logger = pino();

app.use((req, res, next) => {
  req.log = logger.child({ requestId: crypto.randomUUID() });
  req.log.info({ method: req.method, path: req.path }, 'request received');
  next();
});
```

On the frontend, an error boundary combined with a reporting service (Sentry is the common choice) catches render-time exceptions React itself would otherwise only log to the browser console, invisible to the team. Attaching the same request ID pattern — generated client-side and sent as a header on each API call — lets a frontend error report be traced directly to the matching backend log line, which is often the difference between a five-minute and a five-hour debugging session.

```
<Sentry.ErrorBoundary fallback={<ErrorPage />}>
  <App />
</Sentry.ErrorBoundary>
```

21. Worked example: adding a comments feature end to end

To tie the previous chapters together, this section walks through adding one new feature — comments on a task — touching every layer of the stack in sequence, the way a real ticket would.

1. Backend: schema

```
const commentSchema = new mongoose.Schema({
  task: { type: mongoose.Schema.Types.ObjectId, ref: 'Task', required: true },
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  body: { type: String, required: true, maxlength: 1000 },
}, { timestamps: true });

commentSchema.index({ task: 1, createdAt: 1 });
module.exports = mongoose.model('Comment', commentSchema);
```

2. Backend: route

```
router.post('/tasks/:taskId/comments', requireAuth, asyncHandler(async (req, res) => {
  const comment = await Comment.create({
    task: req.params.taskId,
    author: req.userId,
    body: req.body.body,
  });
  io.to(`task:${req.params.taskId}`).emit('comment-added', comment);
  res.status(201).json(comment);
}));
```

3. Frontend: fetching and posting comments

```
function useComments(taskId) {
  const { data, refetch } = useFetch(`/tasks/${taskId}/comments`);

  async function addComment(body) {
    await api.post(`/tasks/${taskId}/comments`, { body });
    refetch();
  }

  return { comments: data ?? [], addComment };
}
```

4. Frontend: real-time updates from other users

```
useEffect(() => {
  const socket = io(import.meta.env.VITE_API_URL);
  socket.emit('join-room', `task:${taskId}`);
  socket.on('comment-added', () => refetch());
  return () => socket.disconnect();
}, [taskId]);
```

Notice how every earlier chapter shows up here: the Mongoose schema and index from chapter 9, the auth middleware from chapter 6, asyncHandler from chapter 12, the WebSocket room pattern from chapter 11, and the useFetch hook from chapter 4. A single feature this small is a useful sanity check that the pieces of the stack actually compose the way each chapter assumed they would.

This guide has walked through the full path from an empty folder to a deployed, authenticated, real-time React and Node.js application. The recurring theme across every chapter is that the frontend and backend agree on a contract — request shapes, error shapes, auth headers, event names — and that most integration bugs come from that contract drifting out of sync rather than from either side being wrong in isolation.